XAVI BOSCH

DESARROLLO DE PRODUCTO INTERACTIVO · PERFIL JUNIOR

bosch.xavii@gmail.com · Barcelona, España · +34 686 585 368
boschwebs.website · github.com/xavibosch · linkedin.com/in/xavier-bosch-galilea-9b3876410



PERFIL PROFESIONAL

Estudiante de segundo curso del grado en Diseño y Creación de Productos Interactivos en La Salle URL y desarrollador autodidacta, con diez productos construidos de principio a fin y varios desplegados en uso diario. Stack principal en Python, Swift y TypeScript sobre React, con Firebase como backend y visión por computador con OpenCV y MediaPipe. Escribo el modelo de datos y la interfaz a la vez, y no doy nada por verificado hasta probarlo en el dispositivo real.

PROYECTOS DESTACADOS

Betsy `github.com/xavibosch/betsy-ios`

Aplicación iOS de apuestas sociales entre amigos con puntos virtuales y sin dinero convertible.

STACK: **Swift, SwiftUI, Cloud Firestore, Firebase Security Rules, Xcode**

- Diseñé el modelo de datos y escribí las reglas de seguridad de Firestore en paralelo a las vistas SwiftUI, lo que destapó una condición de carrera que permitía a dos usuarios liquidar la misma apuesta. Trasladé la autoridad de escritura al servidor y dejé la interfaz solo como solicitante.
- Definí los usuarios objetivo, mapeé el customer journey y ejecuté pruebas con usuarios reales antes de construir las pantallas, rediseñando la interfaz a partir de los resultados y no de mi criterio.

Gesture TV `github.com/xavibosch/gesture-tv`

Control completo de un televisor por gestos y voz, para una persona que no puede usar el mando.

STACK: **Python, OpenCV, MediaPipe, Vosk, Wake on LAN**

- Desarrollé un pipeline de visión por computador en tiempo real con OpenCV y MediaPipe que detecta la mano por zonas de pantalla, más reconocimiento de voz sin conexión con Vosk para el resto de comandos.
- Rediseñé la arquitectura para eliminar el acceso completo por shell y sustituirlo por Wake on LAN y un conjunto acotado de comandos, reduciendo la superficie de ataque. Ningún fotograma de la cámara llega a escribirse en disco.

Reps `github.com/xavibosch/reps`

Panel de entrenamiento manejado con la mano en el aire, sin tocar ninguna pantalla entre series.

STACK: **React, TypeScript, Vite, MediaPipe, Electron, Spotify Web API**

- Desarrollé el seguimiento de manos y una máquina de dos estados, activo y en espera, en React. Lo empaqueté como aplicación de escritorio con Electron e integré la API de Spotify para el control de reproducción.
- Diagnostiqué un desfase de varios centímetros entre el cursor de mano y la detección de pulsación. Un ancestro con CSS transform pasa a ser el bloque contenedor de los descendientes con position fixed, así que el cursor se posicionaba en el marco escalado mientras el hit test leía el viewport real. Sacarlo de ese marco devolvió ambos al mismo sistema de coordenadas.

STACK TECNOLÓGICO

Lenguajes: Python, Swift, TypeScript, JavaScript, Java, HTML, CSS

Frameworks y librerías: SwiftUI, React, Electron, Vite, OpenCV, MediaPipe, Vosk, Core Haptics, PWA

Herramientas, bases de datos y cloud: Git, GitHub, Firebase con Cloud Firestore, Realtime Database y Security Rules, Vercel, MQTT, Ollama, Xcode, Figma, Arduino, ESP32

FORMACIÓN Y CERTIFICACIONES

Grado en Diseño y Creación de Productos Interactivos La Salle, Universitat Ramon Llull, Barcelona. 2025 a la actualidad.
Primer curso completo, 60 ECTS en programación, orientación a objetos, UML, electrónica y diseño de interfaz. Segundo curso desde septiembre de 2026.

Inglés C1 avanzado

Desarrollo autodidacta
Diez productos propios diseñados y construidos de principio a fin, entre aplicaciones iOS, web y sistemas que combinan software con hardware.

COMPETENCIAS CLAVE

Resolución de problemas lógicos: Aíslo los componentes antes de integrarlos. Depurar un sistema montado con motores, sensores y radio a la vez costó horas y probar cada pieza por separado encontró el fallo en minutos.

Aprendizaje autónomo: Todo el stack de arriba lo aprendí porque un proyecto lo necesitaba, sin curso de por medio.

Trabajo en equipo y diseño de interfaces: En un proyecto en pareja fijamos los contratos entre capas antes de escribir el código de debajo, lo que eliminó los bloqueos mutuos.

Criterio de producto: Elimino una función cuando es ambigua por construcción en lugar de seguir ajustándola.